

運用 ADK、AlloyDB 和 Gemini，以 Java 建構功能強大的 E2E AI 代理程式應用程式！

來源：<https://codelabs.developers.google.com/patent-search-java-adk?hl=zh-tw>

步驟數：13

透過這個程式碼研究室，我們將使用 Java 和 Agent Development Kit (ADK)、AlloyDB 和 Gemini 建構功能強大、具狀態且端對端的專利分析代理程式！

目錄

1. [總覽](#)
2. [事前準備](#)
3. [資料庫設定](#)
4. [資料擷取](#)
5. [為專利資料建立嵌入](#)
6. [執行向量搜尋](#)
7. [將資料庫邏輯帶到無伺服器網頁](#)
8. [使用 Java ADK 建構代理](#)
9. [本機 CLI 測試](#)
10. [部署至 Cloud Run](#)
11. [使用網頁版 UI 進行測試](#)
12. [清除所用資源](#)
13. [恭喜](#)

1. 總覽

在不同產業中，情境搜尋都是應用程式的核心功能。檢索增強生成技術採用生成式 AI 輔助的檢索機制，長期以來一直是這項重要技術演進的關鍵推手。生成模型具有大型脈絡窗口和令人驚豔的輸出品質，正在改變 AI。RAG 提供系統化方式，可將背景資訊插入 AI 應用程式和代理程式，讓這些程式/代理程式以結構化資料庫或各種媒體的資訊為基礎。這類情境資料對於釐清事實和確保輸出內容準確性至關重要，但這些結果的準確度如何？您的業務是否高度依賴這些情境比對和關聯性的準確度？那麼這個專案一定會讓您感到有趣！

現在想像一下，如果我們能運用生成模型的力量，建構互動式代理，根據這類重要資訊做出自主決策，並以事實為依據，會怎麼樣？這就是我們今天要建構的內容。我們將使用 Agent Development Kit，在 AlloyDB 中透過進階 RAG 技術，為專利分析應用程式建構端對端 AI 代理程式應用程式。

專利分析代理程式可協助使用者找出與搜尋文字相關的專利，並在使用者要求時，針對所選專利提供簡明扼要的說明和額外詳細資料。想知道如何操作嗎？現在就開始吧！

目標

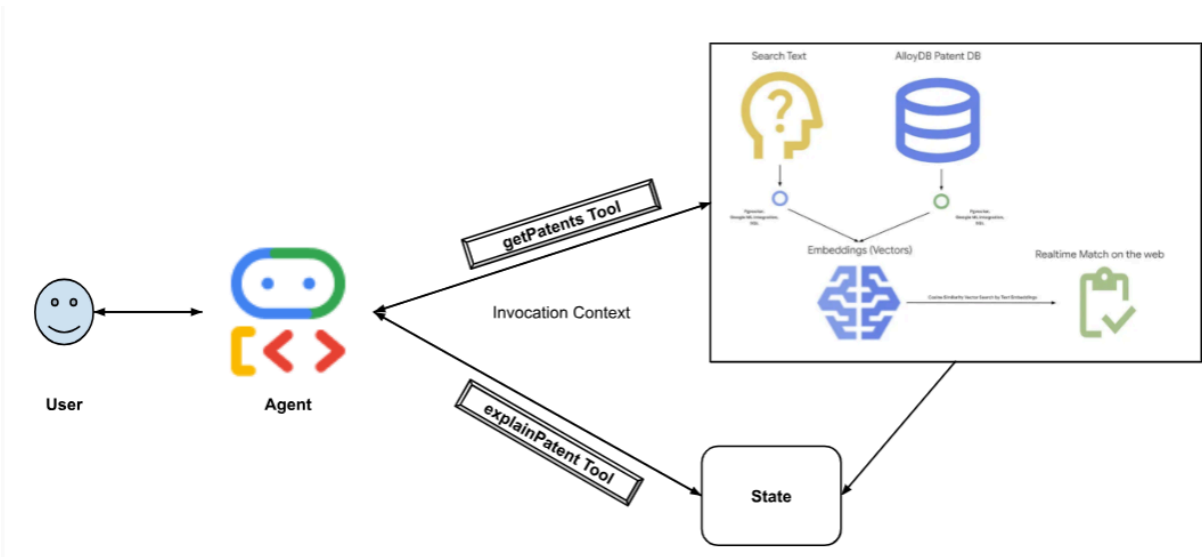
目標很簡單，讓使用者根據文字說明搜尋專利，然後從搜尋結果中取得特定專利的詳細說明，而這一切都是透過以 Java ADK、AlloyDB、向量搜尋 (搭配進階索引)、Gemini 建構的 AI 代理程式完成，且整個應用程式會以無伺服器形式部署在 Cloud Run 上。

建構項目

本實驗室的學習內容包括：

- 1. 建立 AlloyDB 執行個體並載入專利公開資料集資料
- 2. 使用 ScaNN 和 Recall 評估功能，在 AlloyDB 中實作進階向量搜尋
- 3. 使用 Java ADK 建立代理
- 4. 在 Java 無伺服器 Cloud Functions 中實作資料庫伺服器端邏輯
- 5. 在 Cloud Run 部署及測試代理程式

下圖說明實作時的資料流和步驟。



High level diagram representing the flow of the Patent Search Agent with AlloyDB & ADK

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

步驟 2 · 約 0 分鐘

2. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 啟用必要的 API。您可以在 Cloud Shell 終端機中使用 gcloud 指令：

```
gcloud services enable alloydb.googleapis.com compute.googleapis.com  
cloudresourcemanager.googleapis.com servicenetworking.googleapis.com run.googleapis.com  
cloudbuild.googleapis.com cloudfunctions.googleapis.com aiplatform.googleapis.com
```

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 3 · 約 0 分鐘

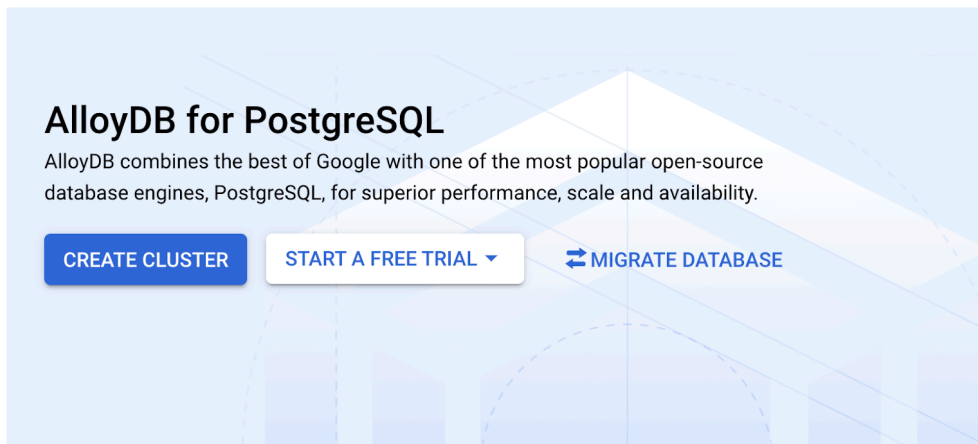
3. 資料庫設定

在本實驗室中，我們將使用 AlloyDB 做為專利資料的資料庫。並使用「叢集」保存所有資源，例如資料庫和記錄檔。每個叢集都有一個「主要執行個體」，可做為資料的存取點。資料表會保存實際資料。

我們來建立 AlloyDB 叢集、執行個體和資料表，以便載入專利資料集。

建立叢集和執行個體

1. 在 Cloud 控制台中前往 AlloyDB 頁面。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 選取該頁面中的「建立叢集」：



3. 畫面上會顯示類似下方的內容。使用下列值建立叢集和執行個體 (如果您要從存放區複製應用程式程式碼，請確保值相符)：

- 叢集 ID：「`vector-cluster`」
- password：「`alloydb`」
- PostgreSQL 15 / 最新建議版本
- Region：「`us-central1`」
- 網路：「`default`」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 選取預設網路後，你會看到如下畫面。

選取「設定連線」。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 然後選取「使用系統自動分配的 IP 範圍」並繼續。確認資訊後，選取「建立連結」。

✓ **Enable Service Networking API**

2 **Allocate an IP range**

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range ▼

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 **Create a connection**

CREATE CONNECTION

CANCEL

6. 設定網路後，即可繼續建立叢集。按一下「CREATE CLUSTER」(建立叢集)，完成叢集設定，如下所示：

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-cluster-primary

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

- ☐ Single zone
In case of outage, no failover. Not recommended for production.
- ☒ Multiple zones (Highly available)
Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

- ☒ Enable Private IP (always enabled)
Connect from applications and clients within the associated network. [Learn more](#)

CREATE CLUSTER

CANCEL

請務必變更執行個體 ID (您可以在設定叢集 / 執行個體時找到)，然後

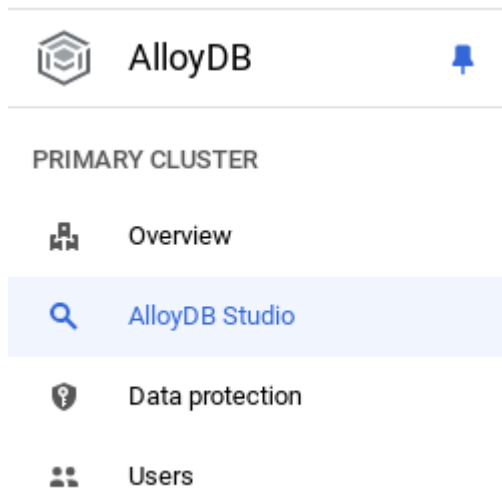
vector-instance。如果無法變更，請記得在所有後續參照中使用執行個體 ID。

請注意，建立叢集約需 10 分鐘。成功後，畫面上會顯示您剛建立的叢集總覽。

步驟 4 · 約 0 分鐘

4. 資料擷取

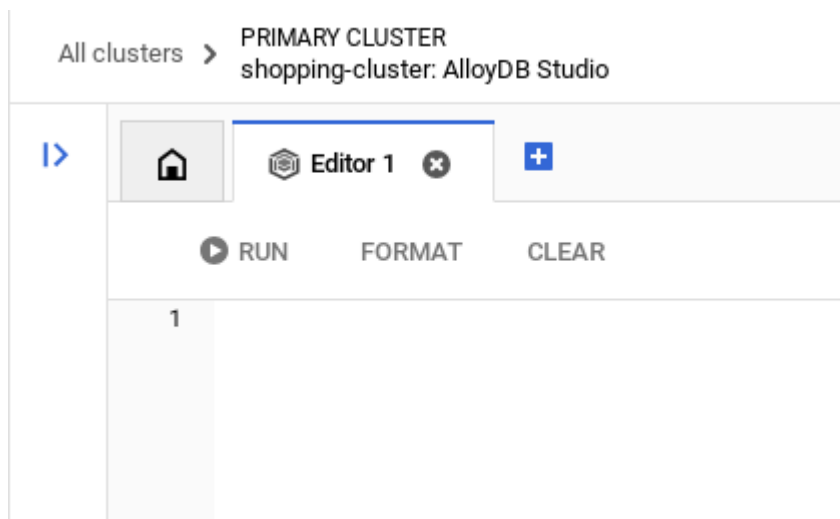
現在要新增包含商店資料的表格。前往 AlloyDB，選取主要叢集，然後選取 AlloyDB Studio：



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

啟用擴充功能

我們會使用 `pgvector` 和 `google_ml_integration` 擴充功能建構這個應用程式。[pgvector 擴充功能](#)可讓您儲存及搜尋向量嵌入。[google_ml_integration](#) 擴充功能提供多種函式，可存取 Vertex AI 預測端點，並在 SQL 中取得預測結果。執行下列 DDL，[啟用](#)這些擴充功能：

```
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;  
CREATE EXTENSION IF NOT EXISTS vector;
```

如要查看資料庫已啟用的擴充功能，請執行下列 SQL 指令：

```
select extname, extversion from pg_extension;
```

建立資料表

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
CREATE TABLE patents_data ( id VARCHAR(25), type VARCHAR(25), number VARCHAR(20), country  
VARCHAR(2), date VARCHAR(20), abstract VARCHAR(300000), title VARCHAR(100000), kind  
VARCHAR(5), num_claims BIGINT, filename VARCHAR(100), withdrawn BIGINT, abstract_embeddings  
vector(768)) ;
```

abstract_embeddings 欄可儲存文字的向量值。

授予權限

執行下列陳述式，授予「embedding」函式的執行權：

```
GRANT EXECUTE ON FUNCTION embedding TO postgres;
```

為 AlloyDB 服務帳戶授予 Vertex AI 使用者角色

在 **Google Cloud IAM** 控制台中，授予 AlloyDB 服務帳戶 (格式如下：service-**<<PROJECT_NUMBER>>**@gcp-sa-alloydb.iam.gserviceaccount.com) 「Vertex AI 使用者」角色存取權。PROJECT_NUMBER 會顯示您的專案編號。

或者，您也可以從 Cloud Shell 終端機執行下列指令：

```
PROJECT_ID=$(gcloud config get-value project)  
  
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member="serviceAccount:service-$(gcloud projects describe $PROJECT_ID --  
format="value(projectNumber)")@gcp-sa-alloydb.iam.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

將專利資料載入資料庫

我們將使用 BigQuery 上的 [Google 專利公開資料集](#) 做為資料集。我們將使用 AlloyDB Studio 執行查詢。資料會匯入這個 `insert scripts sql` 檔案 (位於這個 [存放區](#))，我們會執行這個檔案來載入專利資料。

1. 在 Google Cloud 控制台中，開啟「AlloyDB」[AlloyDB](#) 頁面。
2. 選取新建立的叢集，然後按一下執行個體。
3. 在 AlloyDB 導覽選單中，按一下「AlloyDB Studio」。使用憑證登入。
4. 按一下右側的「新增分頁」圖示，開啟新分頁。

5. 依序複製並執行 `insert_scripts1.sql` , `insert_script2.sql` , `insert_scripts3.sql` , 和 `insert_scripts4.sql` 檔案中的 `insert` 查詢陳述式。您可以執行 10 到 50 個插入陳述式副本，快速展示這個用途。

如要執行，請按一下 **執行**。查詢結果會顯示在「結果」表格中。

步驟 5 · 約 0 分鐘

5. 為專利資料建立嵌入

首先，請執行下列範例查詢，測試嵌入函式：

```
SELECT embedding('text-embedding-005', 'AlloyDB is a managed, cloud-hosted SQL database service.');
```

這應該會傳回查詢中範例文字的嵌入向量，看起來像是浮點數陣列。如下所示：

The screenshot shows a SQL query execution interface. At the top, there are buttons for 'RUN', 'FORMAT', and 'CLEAR'. Below the buttons, the query is displayed: `1 SELECT embedding('textembedding-gecko@001', 'AlloyDB is a managed, cloud-hosted SQL database service.');`. The results section is titled 'RESULTS' and shows a single row with a column named 'embedding'. The value in this column is a long array of floating-point numbers: `[-0.029143702,-0.0071177855,-0.024524968,0.053128935,0.025651122,-0.04028945,-0.0009277421,0.030361904,-0.017621059,0.003031713,0.022722546,-0.00539576,0.044950195,-0.023410132,0.01799109,0.030719345,-0.047324233,-0.021004232,-0.039721053,0.017794741,-0.09268643,-0.061628487,0.0009791466,0.01794383,...`. Below the results, there is a caption: 'Query Result for the Embedding Function Call'.

更新 `abstract_embeddings` 向量欄位

如果需要為專利摘要產生嵌入，請使用下列 DML 更新資料表中的專利摘要，並加入對應的嵌入。但在我們的案例中，插入陳述式已包含每個摘要的這些嵌入內容，因此您不需要呼叫 `embeddings()` 方法。

```
UPDATE patents_data set abstract_embeddings = embedding('text-embedding-005', abstract);
```

6. 執行向量搜尋

現在資料表、資料和嵌入都已準備就緒，讓我們對使用者搜尋文字執行即時向量搜尋。您可以執行下列查詢來測試這項功能：

```
SELECT id || ' - ' || title as title FROM patents_data ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'Sentiment Analysis')::vector LIMIT 10;
```

在這項查詢中，

1. 使用者搜尋的文字為「情緒分析」。
2. 我們會在 embedding() 方法中，使用 text-embedding-005 模型將其轉換為嵌入。
3. 「<=>」代表使用餘弦相似度距離法。
4. 我們會將嵌入方法的結果轉換為向量型別，使其與資料庫中儲存的向量相容。
5. LIMIT 10 代表我們選取與搜尋文字最接近的 10 個相符項目。

AlloyDB 可將向量搜尋 RAG 提升至全新境界：

我們推出了許多新功能，以下是其中兩項以開發人員為主的服務：

1. 內嵌篩選
2. 召回評估人員

內嵌篩選

先前開發人員必須執行 Vector Search 查詢，並處理篩選和召回率作業。AlloyDB 查詢最佳化工具會選擇如何執行含篩選器的查詢。內嵌篩選是全新的查詢最佳化技術，可讓 AlloyDB 查詢最佳化工具同時評估中繼資料篩選條件和向量搜尋，並運用向量索引和中繼資料欄的索引。這項功能可提升召回率效能，讓開發人員充分運用 AlloyDB 的現成功能。

內嵌篩選器最適合用於中等選擇性的情況。AlloyDB 搜尋向量索引時，只會計算符合中繼資料篩選條件的向量距離 (查詢中通常在 WHERE 子句中處理的功能篩選器)。這類查詢的效能大幅提升，可補足[篩選後或篩選前](#)的優勢。

1. 安裝或更新 pgvector 擴充功能

```
CREATE EXTENSION IF NOT EXISTS vector WITH VERSION '0.8.0.google-3';
```

如果已安裝 pgvector 擴充功能，請將向量擴充功能升級至 0.8.0.google-3 以上版本，即可使用回想評估工具。

```
ALTER EXTENSION vector UPDATE TO '0.8.0.google-3';
```

只有在向量擴充功能版本低於 **0.8.0.google-3** 時，才需要執行這個步驟。

重要注意事項：如果列數少於 100，就不需要建立 ScaNN 索引，因為這類索引不適用於列數較少的情況。如果是這種情況，請略過下列步驟。

2. 如要建立 ScaNN 索引，請安裝 alloydb_scann 擴充功能。

```
CREATE EXTENSION IF NOT EXISTS alloydb_scann;
```

3. 請先執行 Vector Search 查詢，但不要啟用索引和內嵌篩選器：

```
SELECT id || ' - ' || title as title FROM patents_data
WHERE num_claims >= 15
ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'Sentiment
Analysis')::vector LIMIT 10;
```

結果應如下所示：

```
1 --explain analyze
2 SELECT id || ' - ' || title as title, abstract FROM patents_data
3 where num_claims >= 50
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')
::vector LIMIT 10;
```

RESULTS		EXPORT		LEARN
title	abstract			
7813618 - Editing of recorded media	Various embodiments provide a system and meth...			
7809215 - Contextual information encoded in a fo...	Embodiments include a method, a manual device,...			
7181978 - Method, apparatus and system for fore...	A method of forecasting strength of a chemically ...			
8792865 - Method and apparatus for adjusting pa...	Systems and methods are provided for mitigating...			
6235494 - Substrates for assessing mannan-bind...	Provided herein are assays for measuring in vivo l...			
7818325 - Serving geospatially organized flat file ...	A flat file data organization technique is used for ...			
8826453 - Content provider with multi-device sec...	Methods and systems for providing access to co...			
Rows per page:		20	1 - 10 of 10	

4. 對其執行 Explain Analyze (不含索引或內嵌篩選)：

```
1 explain analyze
2 SELECT id || ' - ' || title as title, abstract FROM patents_data
3 where num_claims >= 15
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')
::vector LIMIT 10;
```

RESULTS	EXPORT		LEARN
Sort Key: ((abstract_embeddings <=> '[-0.007830657,-0.012530749,-0.005363215,-0.009413715,0.077335395...			
Sort Method: top-N heapsort Memory: 42kB			
-> Seq Scan on patents_data (cost=0.00..280.80 rows=1006 width=744) (actual time=0.052..2.210 rows=487 loops=1)			
Filter: (num_claims >= 15)			
Rows Removed by Filter: 483			
Planning Time: 55.481 ms			
Execution Time: 2.422 ms			

執行時間為 2.4 毫秒

5. 我們在 num_claims 欄位上建立一般索引，以便依該欄位篩選：

```
CREATE INDEX idx_patents_data_num_claims ON patents_data (num_claims);
```

6. 現在為專利搜尋應用程式建立 ScaNN 索引。從 AlloyDB Studio 執行下列指令：

```
CREATE INDEX patent_index ON patents_data
USING scann (abstract_embeddings cosine)
WITH (num_leaves=32);
```

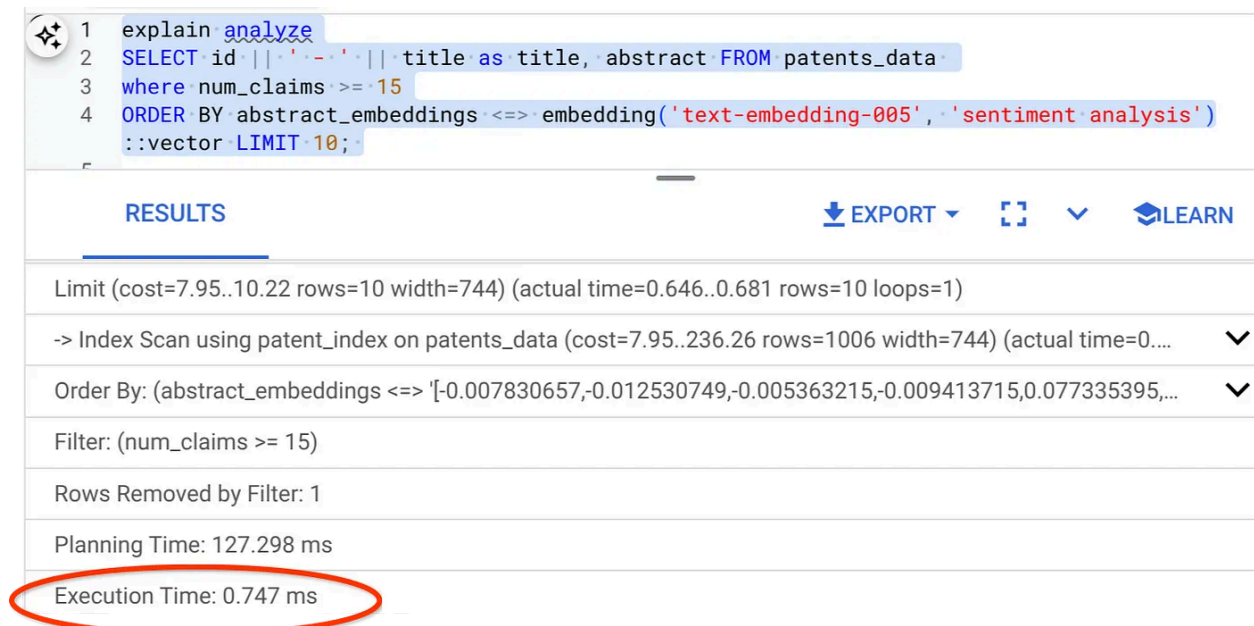
重要注意事項： (num_leaves=32) 適用於超過 1000 列的完整資料集。如果資料列數少於 100，就不需要建立索引，因為索引不適用於較少的資料列。

7. 在 ScaNN 索引中啟用內嵌篩選：

```
SET scann.enable_inline_filtering = on
```

8. 現在，我們來執行相同的查詢，但加入篩選條件和向量搜尋：

```
SELECT id || ' - ' || title as title FROM patents_data
WHERE num_claims >= 15
ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'Sentiment
Analysis')::vector LIMIT 10;
```



```
1 explain analyze
2 SELECT id || ' - ' || title as title, abstract FROM patents_data
3 where num_claims >= 15
4 ORDER BY abstract_embeddings <=> embedding('text-embedding-005', 'sentiment analysis')
::vector LIMIT 10;
```

RESULTS [EXPORT](#) [SQL](#) [LEARN](#)

Limit (cost=7.95..10.22 rows=10 width=744) (actual time=0.646..0.681 rows=10 loops=1)
-> Index Scan using patent_index on patents_data (cost=7.95..236.26 rows=1006 width=744) (actual time=0....
Order By: (abstract_embeddings <=> '[0.007830657,-0.012530749,-0.005363215,-0.009413715,0.077335395,...
Filter: (num_claims >= 15)
Rows Removed by Filter: 1
Planning Time: 127.298 ms
Execution Time: 0.747 ms

如您所見，相同的向量搜尋執行時間大幅縮短。Vector Search 上的內嵌篩選 ScaNN 索引，讓這一切成為可能！

接著，我們來評估啟用 ScaNN 的向量搜尋召回率。

召回評估人員

相似性搜尋的召回率是指從搜尋中擷取的相關例項百分比，也就是真陽性數。這是最常用的搜尋品質評估指標。召回率損失的其中一個來源，是近似最鄰近搜尋 (ANN) 與 k (精確) 最鄰近搜尋 (kNN) 之間的差異。AlloyDB 的 ScaNN 等向量索引會實作 aNN 演算法，讓您加快大型資料集的向量搜尋速度，但召回率會略有損失。現在，AlloyDB 可讓您直接在資料庫中，針對個別查詢評估這項取舍，並確保這項取舍長期穩定。您可以根據這項資訊更新查詢和索引參數，以獲得更出色的結果和效能。

您可以使用 `evaluate_query_recall` 函式，找出特定設定的向量索引向量查詢召回率。您可以使用這個函式調整參數，以取得所需的向量查詢召回結果。召回率是搜尋品質的指標，定義為傳回結果中，客觀上最接近查詢向量的百分比。

「evaluate_query_recall」 *evaluate_query_recall* 函式預設為開啟。

重要注意事項：

如果在後續步驟中，HNSW 索引發生權限遭拒錯誤，請暫時略過整個召回評估部分。這可能是因為本程式碼研究室撰寫時，這項功能才剛發布，因此存取權受到限制。

1. 在 ScaNN 索引和 HNSW 索引上設定「啟用索引掃描」旗標：

```
SET scann.enable_indexscan = on
SET hnsw.enable_index_scan = on
```

2. 在 AlloyDB Studio 中執行下列查詢：

```
SELECT
  *
FROM
  evaluate_query_recall($$
    SELECT
      id || ' - ' || title AS title,
      abstract
    FROM
      patents_data
      where num_claims >= 15
    ORDER BY
      abstract_embeddings <=> embedding('text-embedding-005',
        'sentiment analysis')::vector
    LIMIT 25 $$,
    '{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10}',
    ARRAY['scann']);
```

evaluate_query_recall 函式會將查詢做為參數，並傳回查詢的召回率。我使用與檢查效能時相同的查詢做為函式輸入查詢。我已將 ScaNN 新增為索引方法。如需更多參數選項，請參閱[說明文件](#)。

我們使用的這項向量搜尋查詢的召回率：

RESULTS					EXPORT			LEARN
id	query	configurations	recall	ann_execution_time				
1	SELECT id ' - ' title AS title, abstract FROM patents_data where num_claims >= 15 ORDER BY	{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10, "hnsw.ef_search": 1}	0.7	93.12				

我看到 RECALL 為 70%。現在我可以使用這項資訊變更索引參數、方法和查詢參數，並改善這項向量搜尋的召回率！

我將結果集中的資料列數修改為 7 (先前為 10)，發現 RECALL 略有提升，即 86%。

```

7      id || ' - ' || title AS title,
8      abstract
9  FROM
10     patents_data
11     where num_claims >= 15
12 ORDER BY
13     abstract_embeddings <=> embedding('text-embedding-005',
14     'sentiment analysis')::vector
15 LIMIT 7 $$,
16     '{"scann.num_leaves_to_search":1, "scann.pre_reordering_num_neighbors":10, "hns
ef search": 1}'

```

RESULTS

EXPORT

d	query	configurations	recall
1	SELECT id ' - ' titl...	{"scann.num_leaves_to_search":1, "scann...	0.8571428571428571

也就是說，我可以根據使用者的搜尋情境，即時調整使用者看到的相符結果數量，提高相符結果的關聯性。

好了，現在要部署資料庫邏輯，然後繼續處理代理程式！

7. 將資料庫邏輯帶到無伺服器網頁

準備好將這個應用程式帶到網路上嗎？步驟如下：

1. 前往 Google Cloud 控制台的 Cloud Run Functions，按一下「編寫函式」建立新的 Cloud Run 函式，或使用這個連結：<https://console.cloud.google.com/run/create?deploymentType=function>。
2. 選擇「使用內嵌編輯器建立函式」選項，然後開始設定。提供服務名稱「**patent-search**」，並選擇「**us-central1**」做為區域，以及「Java 17」做為執行階段。將「驗證」設為「允許未經驗證的调用」。
3. 在「容器、磁碟區、網路與安全性」部分，按照下列步驟操作，並詳加說明：

前往「網路」分頁：

Cloud Run

Deploy revision to check-order-status (us-central1)

Every change to the service configuration creates an immutable revision. A revision consists of a specific container image, along with other environment settings.

Container(s)

Volumes

Networking

Security

Connect to other Google Cloud services like Google Cloud Storage or Google Cloud Firestore directly from your code. [Learn more](#)

☐

Use HTTP/2 end-to-end
Use if your container is a gRPC streaming server or is able to directly handle requests in HTTP/2 cleartext. [Learn more](#)

☐

Session affinity
Best effort to route requests from the same client to the same container instance.

☒

Connect to a VPC for outbound traffic
Allow your service to send traffic to a Virtual Private Cloud. [How to choose between VPC options?](#)

☐

Send traffic directly to a VPC
Lower latency, higher throughput, uses more IPs. [Learn more](#)

☒

Use Serverless VPC Access connectors
Use VPC connectors to proxy the traffic. [Learn more](#)

Network

default: Serverless VPC Access Connector "alloydb-conn" (in this project)

Access resources on a VPC. [Learn more](#)

Traffic routing

☐

Route only requests to private IPs to the VPC
Traffic to other Cloud Run services might require additional configuration. [Learn more](#)

Deploy

Cancel

選取「連線至虛擬私有雲，以傳出流量」，然後選取「使用無伺服器 VPC 存取連接器」

在「網路」下拉式選單下方，按一下「網路」下拉式選單，然後選取「新增虛擬私有雲連接器」選項 (如果您尚未設定預設連接器)，並按照彈出式對話方塊中的操作說明進行：

Create connector

Name *

Region *

us-central1

A region is a specific geographical location where you can run your resources.

Network *

default

Subnet *

Select an unused /28 subnet or create a new one by entering an unused /28 IP range. The VPC Connector will create connector instances on this subnet.

▼ SHOW SCALING SETTINGS

CREATE

CANCEL

提供 VPC 連接器的名稱，並確認區域與執行個體相同。將「網路」值保留為預設值，並將「子網路」設為「自訂 IP 範圍」，IP 範圍為 10.8.0.0 或類似的可用範圍。

展開「顯示縮放設定」，確認設定完全符合下列條件：

Scaling Settings

Minimum instances *

2

The minimum number of instances provisioned at any time. The connector will automatically scale out if more capacity is needed. Minimum number of instances cannot be decreased later. Larger values increase your cost.

Maximum instances *

3

The maximum number of instances provisioned at any time. The connector will not automatically scale out above this value. Maximum number of instances cannot be decreased later. This setting limits your maximum cost.



Connectors don't scale down automatically or manually. Once the connector has reached maximum number of instances it will remain at this number.

Instance type *

f1-micro

Larger instances support higher bandwidth but raise your costs. [Learn more](#)

^ HIDE SCALING SETTINGS

CREATE

CANCEL

按一下「建立」，這個連接器現在應該會列在輸出設定中。

選取新建立的連接器。

選擇透過這個虛擬私有雲連接器轉送所有流量。

依序點選「NEXT」(下一步)和「DEPLOY」(部署)。

- 根據預設，這會將進入點設為「gcfv2.HelloHttpFunction」**gcfv2.HelloHttpFunction**。將 Cloud Run 函式的 HelloHttpFunction.java 和 pom.xml 中的預留位置程式碼，分別替換為「[PatentSearch.java](#)」和「[pom.xml](#)」中的程式碼。將類別檔案名稱變更為 PatentSearch.java。
- 請記得在 Java 檔案中，將 ***** 預留位置和 AlloyDB 連線憑證換成您的值。AlloyDB 憑證是我們在本程式碼研究室一開始使用的憑證。如果使用不同的值，請在 Java 檔案中修改。
- 按一下 [Deploy] (部署)。
- 部署更新後的 Cloud 函式後，您應該會看到產生的端點。複製該值，並在下列指令中取代：

```
PROJECT_ID=$(gcloud config get-value project)
```

```
curl -X POST <<YOUR_ENDPOINT>> \  
  -H 'Content-Type: application/json' \  
  -d '{"search": "Sentiment Analysis"}'
```

大功告成！使用 AlloyDB 資料的 Embeddings 模型執行進階脈絡相似度向量搜尋，就是這麼簡單。

步驟 8 · 約 0 分鐘

8. 使用 Java ADK 建構代理

首先，在編輯器中開始使用 Java 專案。

1. 前往 Cloud Shell 終端機

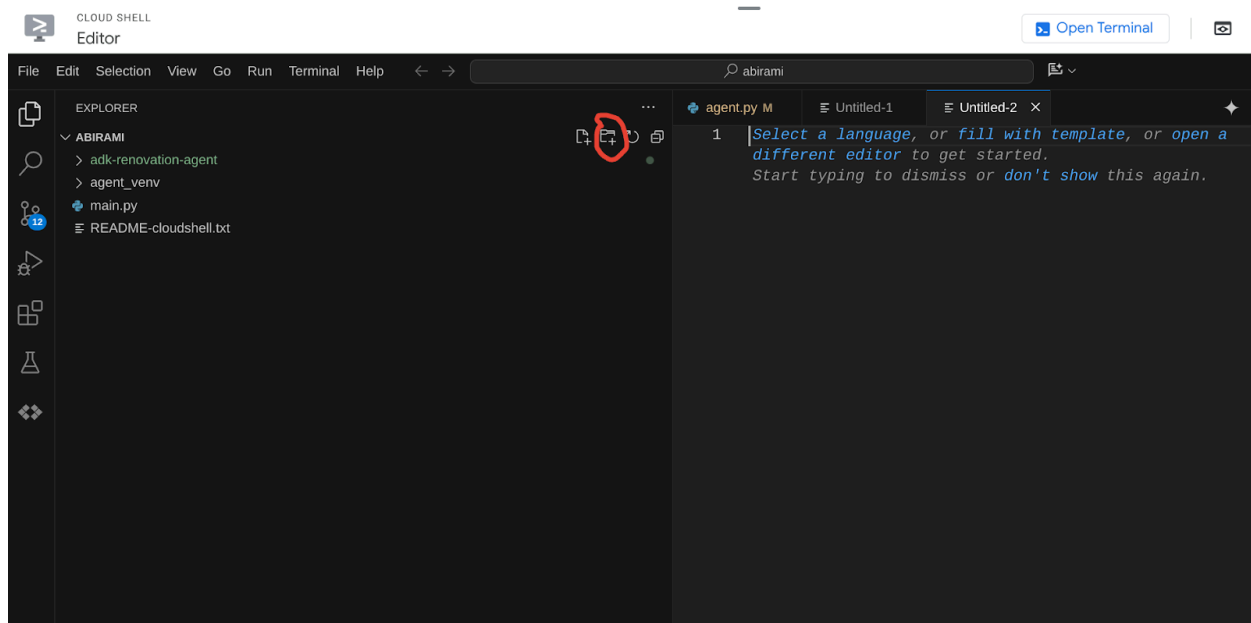
<https://shell.cloud.google.com/?fromcloudshell=true&show=ide%2Cterminal>

2. 在系統提示時授權
3. 按一下 Cloud Shell 控制台頂端的編輯器圖示，即可切換至 Cloud Shell 編輯器



4. 在 Cloud Shell 編輯器控制台中，建立名為「adk-agents」的新資料夾

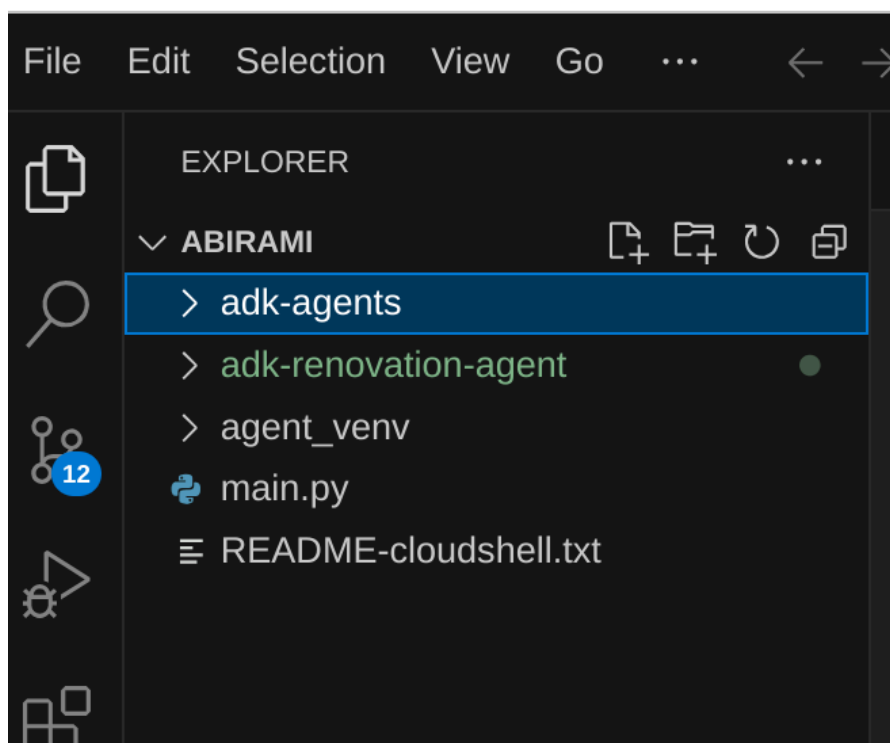
在 Cloud Shell 的根目錄中，按一下「建立新資料夾」，如下所示：



將其命名為「adk-agents」：



CLOUD SHELL Editor



5. 建立下列資料夾結構，並在結構中建立空白檔案，檔案名稱應與下方結構中的名稱相符：

```
adk-agents/  
├── pom.xml  
├── src/  
│   ├── main/  
│   │   ├── java/  
│   │   │   ├── agents/  
│   │   │   └── App.java
```

6. 在另一個分頁中開啟 [GitHub 存放區](#)，然後複製 App.java 和 pom.xml 檔案的原始碼。
7. 如果使用右上角的「在新分頁中開啟」圖示，在新分頁中開啟編輯器，則可以在頁面底部開啟終端機。您可以同時開啟編輯器和終端機，自由操作。
8. 複製完成後，請切換回 Cloud Shell 編輯器控制台
9. 由於我們已建立 Cloud Run 函式，因此不需要從 repo 資料夾複製 Cloud Run 函式檔案。

開始使用 ADK Java SDK

這項作業相當簡單。您主要需要確保複製步驟涵蓋下列項目：

1. 新增依附元件：

在 pom.xml 中加入 google-adk 和 google-adk-dev (適用於網頁介面) 構件。如果您是從存放區複製來源，這些檔案已包含在內，因此不必再加入。您只需要變更 Cloud Run 函式端點，即可反映已部署的端點。本節的後續步驟會說明如何操作。


```

<!-- The ADK core dependency -->
<dependency>
  <groupId>com.google.adk</groupId>
  <artifactId>google-adk</artifactId>
  <version>0.1.0</version>
</dependency>
<!-- The ADK dev web UI to debug your agent -->
<dependency>
  <groupId>com.google.adk</groupId>
  <artifactId>google-adk-dev</artifactId>
  <version>0.1.0</version>
</dependency>

```

請務必參照來源存放區中的 pom.xml，因為應用程式需要其他依附元件和設定才能執行。

2. 設定專案：

請確認 pom.xml 中的 Java 版本 (建議使用 17 以上版本) 和 Maven 編譯器設定正確無誤。您可以將專案設定為下列結構：

```

adk-agents/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── agents/
│   │   │   └── App.java

```

3. 定義代理程式及其工具 (App.java)：

這就是 ADK Java SDK 的神奇之處。我們定義代理、代理的功能 (指令) 和可使用的工具。

如要查看主要代理類別的簡化程式碼片段，請按這裡。如要查看完整專案，請參閱[這裡](#)的專案存放區。

```
// App.java (Simplified Snippets)
package agents;

import com.google.adk.agents.LlmAgent;
import com.google.adk.agents.BaseAgent;
import com.google.adk.agents.InvocationContext;
import com.google.adk.tools.Annotations.Schema;
import com.google.adk.tools.FunctionTool;
// ... other imports

public class App {

    static FunctionTool searchTool = FunctionTool.create(App.class, "getPatents");
    static FunctionTool explainTool = FunctionTool.create(App.class, "explainPatent");

    public static BaseAgent ROOT_AGENT = initAgent();

    public static BaseAgent initAgent() {
        return LlmAgent.builder()
            .name("patent-search-agent")
            .description("Patent Search agent")
            .model("gemini-2.0-flash-001") // Specify your desired Gemini model
            .instruction(
                """
                You are a helpful patent search assistant capable of 2 things:
                // ... complete instructions ...
                """)
            .tools(searchTool, explainTool)
            .outputKey("patents") // Key to store tool output in session state
            .build();
    }

    // --- Tool: Get Patents ---
    public static Map<String, String> getPatents(
        @Schema(name="searchText",description = "The search text for which the user wants to
find matching patents")
        String searchText) {
        try {
            String patentsJson = vectorSearch(searchText); // Calls our Cloud Run Function
            return Map.of("status", "success", "report", patentsJson);
        } catch (Exception e) {
            // Log error
            return Map.of("status", "error", "report", "Error fetching patents.");
        }
    }

    // --- Tool: Explain Patent (Leveraging InvocationContext) ---
    public static Map<String, String> explainPatent(
        @Schema(name="patentId",description = "The patent id for which the user wants to get
more explanation for, from the database")
        String patentId,
        @Schema(name="ctx",description = "The list of patent abstracts from the database from
```

```

which the user can pick the one to get more explanation for")
    InvocationContext ctx) { // Note the InvocationContext
        try {
            // Retrieve previous patent search results from session state
            String previousResults = (String) ctx.session().state().get("patents");
            if (previousResults != null && !previousResults.isEmpty()) {
                // Logic to find the specific patent abstract from 'previousResults' by 'patentId'
                String[] patentEntries = previousResults.split("\n\n\n\n");
                for (String entry : patentEntries) {
                    if (entry.contains(patentId)) { // Simplified check
                        // The agent will then use its instructions to summarize this 'report'
                        return Map.of("status", "success", "report", entry);
                    }
                }
            }
            return Map.of("status", "error", "report", "Patent ID not found in previous
search.");
        } catch (Exception e) {
            // Log error
            return Map.of("status", "error", "report", "Error explaining patent.");
        }
    }

    public static void main(String[] args) throws Exception {
        InMemoryRunner runner = new InMemoryRunner(ROOT_AGENT);
        // ... (Session creation and main input loop - shown in your source)
    }
}

```

重點 ADK Java 程式碼元件：

1. **LlmAgent.builder()**: 用於設定代理程式的 Fluent API。
2. **.instruction(...)**: 提供 LLM 的核心提示和指南，包括何時使用哪種工具。
3. **FunctionTool.create(App.class, "methodName")**: 輕鬆將 Java 方法註冊為代理程式可叫用的工具。方法名稱字串必須與實際的公開靜態方法相符。
4. **@Schema(description = ...)**: 註解工具參數，協助 LLM 瞭解各工具預期的輸入內容。這項說明對於正確選取工具和填寫參數至關重要。
5. **InvocationContext ctx**: 自動傳遞至工具方法，可存取工作階段狀態 (ctx.session().state())、使用者資訊等。
6. **.outputKey("patents")**: 工具傳回資料時，ADK 可以自動將資料儲存在工作階段狀態中，並使用這個鍵。這就是 explainPatent 如何存取 getPatents 的結果。
7. **VECTOR_SEARCH_ENDPOINT**: 這個變數包含專利搜尋用例中，使用者情境式問答的核心功能邏輯。
8. 行動項目：實作前一節的 Java Cloud Run 函式步驟後，您需要設定更新後的已部署端點值。
9. **searchTool**: 與使用者互動，從專利資料庫中找出與使用者搜尋文字相關的專利。
10. **explainTool**: 要求使用者提供特定專利，以便深入瞭解。然後摘要專利摘要，並根據專利詳細資料回答使用者的其他問題。

重要事項：請務必將 **VECTOR_SEARCH_ENDPOINT** 變數替換為已部署的 CRF 端點。

運用 InvocationContext 進行有狀態的互動

建構實用代理的關鍵功能之一，就是管理多輪對話的狀態。ADK 的 InvocationContext 可簡化這項作業。

在 App.java 中：

1. 定義 initAgent() 時，我們會使用 .outputKey("patents")。這會告知 ADK，當工具 (例如 getPatents) 在報表欄位中傳回資料時，該資料應儲存在工作階段狀態中，並使用「patents」鍵。
2. 在 explainPatent 工具方法中，我們會注入 InvocationContext ctx：

```
public static Map<String, String> explainPatent(
    @Schema(description = "...") String patentId, InvocationContext ctx) {
    String previousResults = (String) ctx.session().state().get("patents");
    // ... use previousResults ...
}
```

這樣一來，explainPatent 工具就能存取 getPatents 工具在前一回合擷取的專利清單，讓對話保持有狀態並連貫。

步驟 9 · 約 0 分鐘

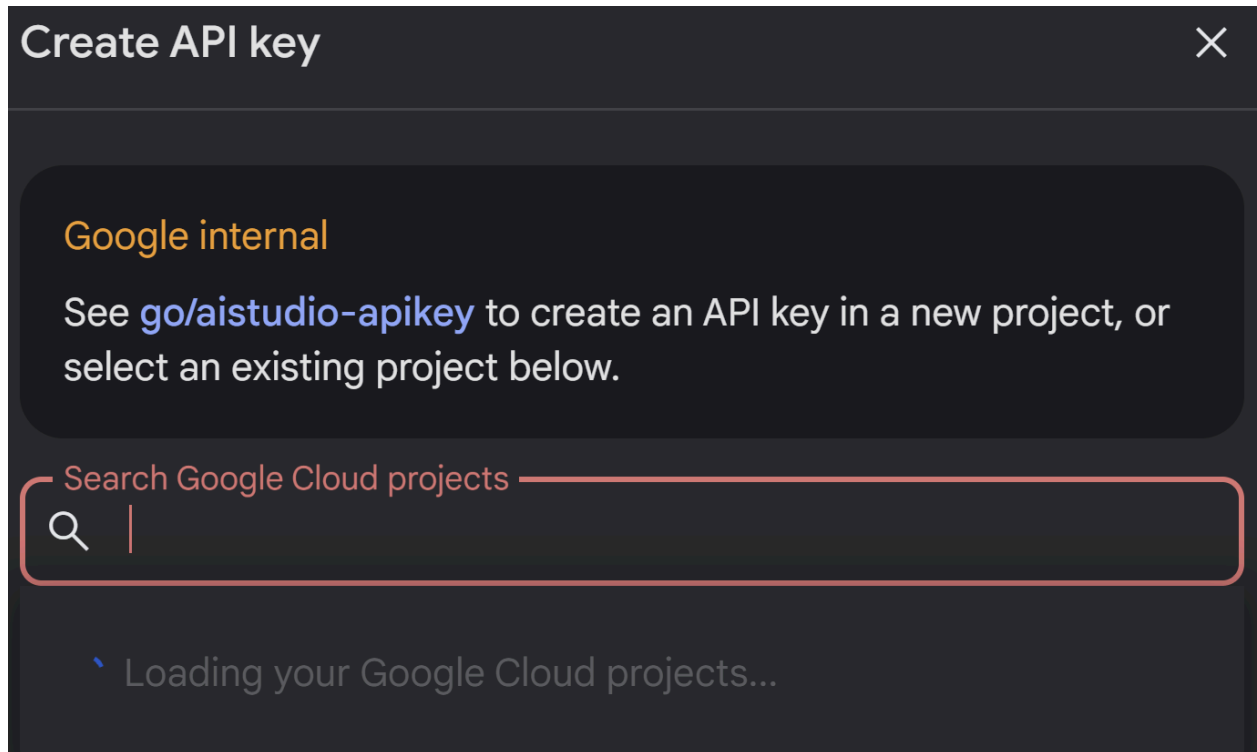
9. 本機 CLI 測試

定義環境變數

您需要匯出兩項環境變數：

1. 從 AI Studio 取得的 Gemini 金鑰：

如要這麼做，請前往 <https://aistudio.google.com/apikey>，取得您要實作這個應用程式的有效 Google Cloud 專案 API 金鑰，並將金鑰儲存在某處：



1. 取得金鑰後，開啟 Cloud Shell 終端機，然後執行下列指令，移至我們剛建立的 adk-agents 新目錄：

```
cd adk-agents
```

2. 這個變數用於指定這次不使用 Vertex AI。

```
export GOOGLE_GENAI_USE_VERTEXAI=FALSE  
export GOOGLE_API_KEY=AiZaSyDF...
```

3. 在 CLI 上執行第一個代理程式

如要啟動第一個代理程式，請在終端機中使用下列 Maven 指令：

```
mvn compile exec:java -DmainClass="agents.App"
```

終端機中會顯示代理程式的互動式回覆。

步驟 10 · 約 0 分鐘

10. 部署至 Cloud Run

將 ADK Java 代理程式部署至 Cloud Run，與部署任何其他 Java 應用程式類似：

1. Dockerfile：建立 Dockerfile 來封裝 Java 應用程式。
2. 建構及推送 Docker 映像檔：使用 Google Cloud Build 和 Artifact Registry。
3. 您只要執行一個指令，即可完成上述步驟並部署至 Cloud Run：

```
gcloud run deploy --source . --set-env-vars GOOGLE_API_KEY=<<Your_Gemini_Key>>
```

同樣地，您會部署 Java Cloud Run 函式 (gcfv2.PatentSearch)。或者，您也可以直接從 Cloud Run 函式控制台，為資料庫邏輯建立及部署 Java Cloud Run 函式。

11. 使用網頁版 UI 進行測試

ADK 隨附實用的網頁式 UI，方便您在本機測試及偵錯代理。在本機執行 `App.java` 時 (例如，如果已設定，則為 `mvn exec:java -Dexec.mainClass="agents.App"`，或只是執行主要方法)，ADK 通常會啟動本機網路伺服器。

ADK 網頁版 UI 可讓您：

1. 傳送訊息給服務專員。
2. 查看事件 (使用者訊息、工具呼叫、工具回應、LLM 回覆)。
3. 检查工作階段狀態。
4. 查看記錄和追蹤記錄。

這在開發期間非常重要，可協助您瞭解代理程式如何處理要求及使用工具。前提是 `pom.xml` 中的 `mainClass` 設為 `com.google.adk.web.AdkWebServer`，且代理程式已向其註冊，或是您正在執行會公開這項資訊的本機測試執行元件。

使用 `InMemoryRunner` 和 `Scanner` 執行 `App.java` 以進行主控台輸入時，您會測試核心代理程式邏輯。Web UI 是獨立元件，可提供更直觀的偵錯體驗，通常用於 ADK 透過 HTTP 服務代理時。

您可以在根目錄中使用下列 Maven 指令，啟動 SpringBoot 本機伺服器：

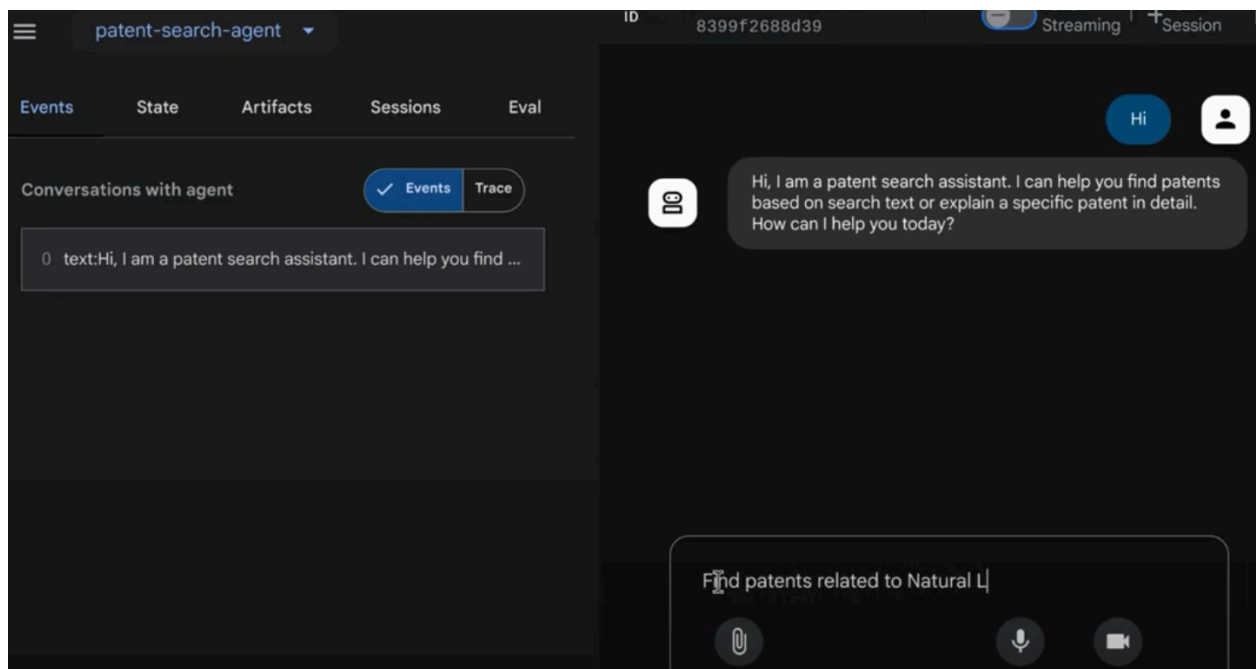
```
mvn compile exec:java -Dexec.args="--adk.agents.source-dir=src/main/java/ --  
logging.level.com.google.adk.dev=TRACE --logging.level.com.google.adk.demo.agents=TRACE"
```

通常可透過上述指令輸出的網址存取介面。如果是已部署的 **Cloud Run**，您應該可以透過已部署的 Cloud Run 連結存取。

您應該可以在互動式介面中看到結果。

請觀看下方影片，瞭解已部署的專利代理人：

[運用 AlloyDB 內嵌搜尋和召回評估，控管專利代理人品質的示範！](#)



12. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中，前往 https://console.cloud.google.com/cloud-resource-manager?utm_campaign=CDR_0x1d2a42f5_default_b419133749&utm_medium=external&utm_source=blog
 2. https://console.cloud.google.com/cloud-resource-manager?utm_campaign=CDR_0x1d2a42f5_default_b419133749&utm_medium=external&utm_source=blog 頁面。
 3. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 4. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 13 · 約 0 分鐘

13. 恭喜

恭喜！您已成功結合 ADK、https://cloud.google.com/alloydb/docs?utm_campaign=CDR_0x1d2a42f5_default_b419133749&utm_medium=external&utm_source=blog、Vertex AI 和 Vector Search 的功能，以 Java 建構專利分析代理程式，並在情境相似性搜尋方面取得重大進展，讓這項技術更具轉變性、效率和意義。

立即開始使用！

ADK 說明文件：[Link to Official ADK Java Docs]

專利分析代理程式原始碼：[Link to your (now public) GitHub Repo]

Java 範例代理程式：[link to the adk-samples repo]

加入 ADK 社群：<https://www.reddit.com/r/agentdevelopmentkit/>

祝您建構代理程式愉快！
